This is **unpublished** documentation of working with Rust and WebAssembly, the published documentation is available on the main Rust and WebAssembly documentation site . Features documented here may not be available in released versions of tooling for Rust and WebAssembly.

Shrinking `.wasm` Code Size

This section will teach you how to optimize your `.wasm` build for a small code size footprint, and how to identify opportunities to change your Rust source such that less `.wasm` code is emitted.

Why Care About Code Size?

When serving a `.wasm` file over the network, the smaller it is, the faster the client can download it. Faster `.wasm` downloads lead to faster page load times, and that leads to happier users.

However, it's important to remember though that code size likely isn't the end-all-be-all metric you're interested in, but rather something much more vague and hard to measure like "time to first interaction". While code size plays a large factor in this measurement (can't do anything if you don't even have all the code yet!) it's not the only factor.

WebAssembly is typically served to users gzip'd so you'll want to be sure to compare differences in gzip'd size for transfer times over the wire. Also keep in mind that the WebAssembly binary format is quite amenable to gzip compression, often getting over 50% reductions in size.

Furthermore, WebAssembly's binary format is optimized for very fast parsing and processing. Browsers nowadays have "baseline compilers" which parses WebAssembly and emits compiled code as fast as wasm can come in over the network. This means that if you're using `instantiateStreaming` the second the Web request is done the WebAssembly module is probably ready to go. JavaScript, on the other hand, can often take longer to not only parse but also get up to speed with JIT compilation and such.

And finally, remember that WebAssembly is also far more optimized than JavaScript for execution speed. You'll want to be sure to measure for runtime comparisons between JavaScript and WebAssembly to factor that in to how important code size is.

All this to say basically don't dismay immediately if your `.wasm` file is larger than expected! Code size may end up only being one of many factors in the end-to-end story. Comparisons between JavaScript and WebAssembly that only look at code size are missing the forest for the trees.

Optimizing Builds for Code Size

There are a bunch of configuration options we can use to get `rustc` to make smaller `.wasm` binaries. In some cases, we are trading longer compile times for smaller `.wasm` sizes. In other cases, we are trading runtime speed of the WebAssembly for smaller code size. We should be cognizant of the trade offs of each option, and in the cases where we trade runtime speed for code size, profile and measure to make an informed decision about whether the trade is worth it.

Compiling with Link Time Optimizations (LTO)

In `Cargo.toml`, add `lto = true` in the `[profile.release]` section:

```
[profile.release]
lto = true
```

This gives LLVM many more opportunities to inline and prune functions. Not only will it make the `.wasm` smaller, but it will also make it faster at runtime! The downside is that compilation will take longer.

Tell LLVM to Optimize for Size Instead of Speed

LLVM's optimization passes are tuned to improve speed, not size, by default. We can change the goal to code size by modifying the `[profile.release]` section in `Cargo.toml` to this:

```
[profile.release]
opt-level = 's'
```

Or, to even more aggressively optimize for size, at further potential speed costs:

```
[profile.release]
opt-level = 'z'
```

Note that, surprisingly enough, `opt-level = "s"` can sometimes result in smaller binaries than `opt-level = "z"`. Always measure!

Use the `wasm-opt` Tool

The Binaryen toolkit is a collection of WebAssembly-specific compiler tools. It goes much further than LLVM's WebAssembly backend does, and using its `wasm-opt` tool to post-process a `.wasm` binary generated by LLVM can often get another 15-20% savings on code size. It will often produce runtime speed ups at the same time!

```
# Optimize for size.
wasm-opt -Os -o output.wasm input.wasm

# Optimize aggressively for size.
wasm-opt -Oz -o output.wasm input.wasm

# Optimize for speed.
wasm-opt -O -o output.wasm input.wasm

# Optimize aggressively for speed.
wasm-opt -O3 -o output.wasm input.wasm
```

Notes about Debug Information

One of the biggest contributors to wasm binary size can be debug information and the `names` section of the wasm binary. The `wasm-pack` tool, however, removes debuginfo by default. Additionally `wasm-opt` removes the `names` section by default unless `-g` is also specified.

This means that if you follow the above steps you should by default not have either debuginfo or the names section in the wasm binary. If, however, you are manually otherwise preserving this debug information in the wasm binary be sure to be mindful of this!

Size Profiling

If tweaking build configurations to optimize for code size isn't resulting in a small enough `.wasm` binary, it is time to do some profiling to see where the remaining code size is coming from.

> ⚡ Just like how we let time profiling guide our speed up efforts, we want to let size profiling guide our code size shrinking efforts. Fail to do this and you risk wasting your own time!

The `twiggy` Code Size Profiler

`twiggy` is a code size profiler that supports WebAssembly as input. It analyzes a binary's call graph to answer questions like:

- Why was this function included in the binary in the first place?
- What is the *retained size* of this function? I.e. how much space would be saved if I removed it and all the functions that become dead code after its removal?

```
$ twiggy top -n 20 pkg/wasm_game_of_life_bg.wasm
 Shallow Bytes │ Shallow % │ Item
───────────────┼───────────┼──────────────────────────────────────────────────────────────────────────────────
          9158 ┊    19.65% ┊ "function names" subsection
          3251 ┊     6.98% ┊ dlmalloc::dlmalloc::Dlmalloc::malloc::h632d10c184fef6e8
          2510 ┊     5.39% ┊ <str as core::fmt::Debug>::fmt::he0d87479d1c208ea
          1737 ┊     3.73% ┊ data[0]
          1574 ┊     3.38% ┊ data[3]
          1524 ┊     3.27% ┊ core::fmt::Formatter::pad::h6825605b326ea2c5
          1413 ┊     3.03% ┊ std::panicking::rust_panic_with_hook::h1d3660f2e339513d
          1200 ┊     2.57% ┊ core::fmt::Formatter::pad_integral::h06996c5859a57ced
          1131 ┊     2.43% ┊ core::str::slice_error_fail::h6da90c14857ae01b
          1051 ┊     2.26% ┊ core::fmt::write::h03ff8c7a2f3a9605
           931 ┊     2.00% ┊ data[4]
           864 ┊     1.85% ┊ dlmalloc::dlmalloc::Dlmalloc::free::h27b781e3b06bdb05
           841 ┊     1.80% ┊ <char as core::fmt::Debug>::fmt::h07742d9f4a8c56f2
           813 ┊     1.74% ┊ __rust_realloc
           708 ┊     1.52% ┊ core::slice::memchr::memchr::h6243a1b2885fdb85
           678 ┊     1.45% ┊ <core::fmt::builders::PadAdapter<'a> as
core::fmt::Write>::write_str::h96b72fb7457d3062
           631 ┊     1.35% ┊ universe_tick
           631 ┊     1.35% ┊
dlmalloc::dlmalloc::Dlmalloc::dispose_chunk::hae6c5c8634e575b8
           514 ┊     1.10% ┊ std::panicking::default_hook::
{{closure}}::hfae0c204085471d5
           503 ┊     1.08% ┊ <&'a T as core::fmt::Debug>::fmt::hba207e4f7abaece6
```

Manually Inspecting LLVM-IR

LLVM-IR is the final intermediate representation in the compiler toolchain before LLVM generates WebAssembly. Therefore, it is very similar to the WebAssembly that is ultimately emitted. More LLVM-IR generally means more `.wasm` size, and if a function takes up 25% of the LLVM-IR, then it generally will take up 25% of the `.wasm`. While these numbers only hold in general, the LLVM-IR has crucial information that is not present in the `.wasm` (because of WebAssembly's lack of a debugging format like DWARF): which subroutines were inlined into a given function.

You can generate LLVM-IR with this `cargo` command:

```
cargo rustc --release -- --emit llvm-ir
```

Then, you can use `find` to locate the `.ll` file containing the LLVM-IR in `cargo`'s `target` directory:

```
find target/release -type f -name '*.ll'
```

References

- LLVM Language Reference Manual

More Invasive Tools and Techniques

Tweaking build configurations to get smaller `.wasm` binaries is pretty hands off. When you need to go the extra mile, however, you are prepared to use more invasive techniques, like rewriting source code to avoid bloat. What follows is a collection of get-your-hands-dirty techniques you can apply to get smaller code sizes.

Avoid String Formatting

`format!`, `to_string`, etc... can bring in a lot of code bloat. If possible, only do string formatting in debug mode, and in release mode use static strings.

Avoid Panicking

This is definitely easier said than done, but tools like `twiggy` and manually inspecting LLVM-IR can help you figure out which functions are panicking.

Panics do not always appear as a `panic!()` macro invocation. They arise implicitly from many constructs, such as:

- Indexing a slice panics on out of bounds indices: `my_slice[i]`
- Division will panic if the divisor is zero: `dividend / divisor`
- Unwrapping an `Option` or `Result`: `opt.unwrap()` or `res.unwrap()`

The first two can be translated into the third. Indexing can be replaced with fallible `my_slice.get(i)` operations. Division can be replaced with `checked_div` calls. Now we only have a single case to contend with.

Unwrapping an `Option` or `Result` without panicking comes in two flavors: safe and unsafe.

The safe approach is to `abort` instead of panicking when encountering a `None` or an `Error`:

```
#[inline]
pub fn unwrap_abort<T>(o: Option<T>) -> T {
    use std::process;
    match o {
        Some(t) => t,
        None => process::abort(),
    }
}
```

Ultimately, panics translate into aborts in `wasm32-unknown-unknown` anyways, so this gives you the same behavior but without the code bloat.

Alternatively, the `unreachable` crate provides an unsafe `unchecked_unwrap` extension method for `Option` and `Result` which tells the Rust compiler to *assume* that the `Option` is `Some` or the `Result` is `Ok`. It is undefined behavior what happens if that assumption does not hold. You really only want to use this unsafe approach when you 110% *know* that the assumption holds, and the compiler just isn't smart enough to see it. Even if you go down this route, you should have a debug build configuration that still does the checking, and only use unchecked operations in release builds.

Avoid Allocation or Switch to `wee_alloc`

Rust's default allocator for WebAssembly is a port of `dlmalloc` to Rust. It weighs in somewhere around ten kilobytes. If you can completely avoid dynamic allocation, then you should be able to shed those ten kilobytes.

Completely avoiding dynamic allocation can be very difficult. But removing allocation from hot code paths is usually much easier (and usually helps make those hot code paths faster, as well). In these cases, replacing the default global allocator with `wee_alloc` should save you most (but not quite all) of those ten kilobytes. `wee_alloc` is an allocator designed for situations where you need *some* kind of allocator, but do not need a particularly fast allocator, and will happily trade allocation speed for smaller code size.

Use Trait Objects Instead of Generic Type Parameters

When you create generic functions that use type parameters, like this:

```
fn whatever<T: MyTrait>(t: T) { ... }
```

Then `rustc` and LLVM will create a new copy of the function for each `T` type that the function is used with. This presents many opportunities for compiler optimizations based on which particular `T` each copy is working with, but these copies add up quickly in terms of code size.

If you use trait objects instead of type parameters, like this:

```
fn whatever(t: Box<MyTrait>) { ... }
// or
fn whatever(t: &MyTrait) { ... }
// etc...
```

Then dynamic dispatch via virtual calls is used, and only a single version of the function is emitted in the `.wasm`. The downside is the loss of the compiler optimization opportunities and the added cost of indirect, dynamically dispatched function calls.

Use the `wasm-snip` Tool

`wasm-snip` replaces a WebAssembly function's body with an `unreachable` instruction. This is a rather heavy, blunt hammer for functions that kind of look like nails if you squint hard enough.

Maybe you know that some function will never be called at runtime, but the compiler can't prove that at compile time? Snip it! Afterwards, run `wasm-opt` again with the `--dce` flag, and all the functions that the snipped function transitively called (which could also never be called at runtime) will get removed too.

This tool is particularly useful for removing the panicking infrastructure, since panics ultimately translate into traps anyways.